

Finite Difference Methods for Options Pricing

Vahan Hovhannisyan

2026

1 Introduction

This tutorial covers finite difference methods for finance and options theory, specifically focusing on finding solutions to the Black-Scholes equation for a European put option. The underlying theory is based on *Numerical Methods in Finance with C++* by M. Capiński and T. Zastawniak, but we will implement the solution in Python.

2 Parabolic Partial Differential Equations

First, we consider the general parabolic differential equation:

$$\frac{\partial v(t, x)}{\partial t} = a(t, x) \frac{\partial^2 v(t, x)}{\partial x^2} + b(t, x) \frac{\partial v(t, x)}{\partial x} + c(t, x) v(t, x) + d(t, x) \quad (1)$$

for $(t, x) \in [0, T] \times [x_l, x_u]$.

The problem includes the following boundary conditions:

- **Terminal boundary condition:** $v(T, x) = f(x)$.
- **Lower boundary condition:** $v(t, x_l) = f_l(t)$.
- **Upper boundary condition:** $v(t, x_u) = f_u(t)$.

3 The Black-Scholes Equation

A typical example of a parabolic PDE is the Black-Scholes equation:

$$\frac{\partial u(t, z)}{\partial t} + \frac{\sigma^2}{2} z^2 \frac{\partial^2 u(t, z)}{\partial z^2} + rz \frac{\partial u(t, z)}{\partial z} - ru(t, z) = 0 \quad (2)$$

Here, $u(t, S(t))$ represents a pricing function for an underlying asset with price $S(t)$ at time t .

This equation is a special case of the general parabolic PDE with the following parameters:

- $a(t, z) = -\frac{\sigma^2}{2} z^2$
- $b(t, z) = -rz$
- $c(t, z) = r$
- $d(t, z) = 0$

For a European put option with an expiry date T and a strike price K , the boundary conditions are:

- $f(z) = u(T, z) = \max\{K - z, 0\}$
- $f_u(t) = u(t, z_u) = 0$
- $f_l(t) = u(t, z_l) = e^{-r(T-t)} K$

To model this, you need data about the underlying asset (the risk-free interest rate and standard deviation) and the option itself (the strike price, time to expiration, and the lower/upper bounds for the spot price).

4 The Explicit Scheme

The explicit finite difference method operates on a finite set of points $\{(t_i, x_j) : i = 0, \dots, i_{\max} \text{ and } j = 0, \dots, j_{\max}\}$.

The grid step sizes are defined as $\Delta t = \frac{T}{i_{\max}}$ and $\Delta x = \frac{x_u - x_l}{j_{\max}}$.

The explicit scheme relies on the following recursive formula:

$$v_{i-1,j} = A_{i,j}v_{i,j-1} + B_{i,j}v_{i,j} + C_{i,j}v_{i,j+1} + D_{i,j} \quad (3)$$

The coefficients for this equation are given by:

- $A_{i,j} = \frac{\Delta t}{\Delta x} \left(\frac{b_{i,j}}{2} - \frac{a_{i,j}}{\Delta x} \right)$
- $B_{i,j} = 1 - \Delta t c_{i,j} + \frac{2\Delta t a_{i,j}}{\Delta x^2}$
- $C_{i,j} = -\frac{\Delta t}{\Delta x} \left(\frac{b_{i,j}}{2} + \frac{a_{i,j}}{\Delta x} \right)$
- $D_{i,j} = -\Delta t d_{i,j}$

Because the values of $v_{i,j}$ can be determined from the boundary conditions, the computation starts from $i = i_{\max}$ and finishes at $i = 0$. The boundary conditions establish that $v_{i_{\max},j} = f_j$, $v_{i-1,0} = f_{l,i-1}$, and $v_{i-1,j_{\max}} = f_{u,i-1}$.

5 The Implicit Scheme

Implicit schemes use different approximations for the partial gradients of v , meaning there is no explicit formula for $v_{i-1,j}$. Instead, the iterative formula is written in matrix form:

$$\mathbf{v}_{i-1} = \mathbf{B}_i^{-1}(\mathbf{A}_i \mathbf{v}_i + \mathbf{w}_i) \quad (4)$$

The vectors and matrices are structured as follows:

- $\mathbf{v}_i$ is a column vector containing the values $v_{i,1}$ through $v_{i,j_{\max}-1}$.
- $\mathbf{A}_i$ is a tridiagonal matrix with the coefficients $A_{i,j}$, $B_{i,j}$, and $C_{i,j}$ along its diagonals.
- $\mathbf{B}_i$ is a tridiagonal matrix with the coefficients $E_{i,j}$, $F_{i,j}$, and $G_{i,j}$ along its diagonals.
- $\mathbf{w}_i$ is a vector that incorporates the boundary conditions along with the $D_{i,j}$ terms.

The scalars used to construct these matrices for the implicit scheme are defined as:

- $A_{i,j} = 0$, $B_{i,j} = 1$, $C_{i,j} = 0$
- $D_{i,j} = -\Delta t d_{i-1,j}$
- $E_{i,j} = -\frac{\Delta t}{\Delta x} \left(\frac{b_{i-1,j}}{2} - \frac{a_{i-1,j}}{\Delta x} \right)$
- $F_{i,j} = 1 + \Delta t c_{i-1,j} - \frac{2\Delta t a_{i-1,j}}{\Delta x^2}$
- $G_{i,j} = \frac{\Delta t}{\Delta x} \left(\frac{b_{i-1,j}}{2} + \frac{a_{i-1,j}}{\Delta x} \right)$

Finally, a related algorithm called the Crank-Nicolson Scheme can be derived by overriding the A through F scalar coefficients defined here.